



RELATÓRIO FINAL DE EXECUÇÃO

Tutorial 2 (AWS): Ingestão com Meltano na Nuvem

Tutorial 2 (AWS): Ingestão com Meltano na Nuvem

RDS PostgreSQL e PokéAPI para arquivos Parquet no Amazon S3

Disciplina: Ecossistema de Big Data

Professor: Leandro Mendes Ferreira

Grupo A

Nome	Matrícula USP
Breno Tostes Garcia	18105650
Gerson Chadi Junior	4360987
Isabella Abreu Comelli	18106391
Fernando Luiz Gomes	18105793
Caetano Sales Carvalho	18105692

São Paulo, 27 de julho de 2026

1. INTRODUÇÃO

O Tutorial 2 propõe a execução de uma solução de ingestão com Meltano na AWS. O processamento ocorre em uma instância EC2, o banco de origem é um RDS PostgreSQL privado e o destino final é um bucket S3 real. A ferramenta Meltano organiza plugins compatíveis com o padrão Singer, permitindo combinar extratores, ou taps, com destinos, ou targets.

Na implementação realizada, o tap-postgres extraiu a tabela public.vendas e o tap-rest-api-msdk consultou a PokéAPI. O target-parquet gravou os resultados em diretórios locais na EC2. Em seguida, a AWS CLI sincronizou somente os arquivos Parquet com os prefixos meltano/vendas e meltano/pokemon do bucket provisionado pelo Tutorial 1.

O trabalho foi conduzido com registro sistemático de comandos, códigos de saída, arquivos de configuração, validações de conteúdo, hashes SHA-256 e capturas de tela. As falhas encontradas foram preservadas como evidência e tratadas como parte do processo de aprendizagem e de melhoria da solução.

2. OBJETIVO E ESCOPO

2.1. Objetivo Geral

Executar e documentar pipelines de ingestão com Meltano em uma infraestrutura AWS, extraindo dados de um RDS PostgreSQL e de uma API REST, convertendo-os para Parquet e publicando-os em um bucket S3.

2.2. Objetivo Específico

- Revalidar a infraestrutura provisionada por Terraform e confirmar a disponibilidade da EC2, do RDS e do S3.
- Preparar um ambiente Python isolado e instalar o Meltano 4.2.1.
- Configurar tap-postgres, tap-rest-api-msdk e target-parquet.
- Extrair e validar os 200 registros da tabela public.vendas.

- Extrair o catálogo disponível na PokéAPI e corrigir a paginação para incluir o primeiro registro.
- Publicar os Parquet no S3 e validar os objetos por leitura, contagem e comparação de hashes.
- Preservar evidências suficientes para auditoria, reprodução e elaboração do relatório final.

2.3.Critério de Aceite

Tabela 2-1: Critérios de aceite utilizados

Critério	Condição de aprovação
Infraestrutura	Terraform sem drift; EC2 running; RDS available; S3 acessível
Instalação	Python, venv e Meltano funcionais
Plugins	Três plugins instalados e invocáveis
PostgreSQL	200 linhas no Parquet
PokéAPI	Quantidade alinhada à API; nomes únicos; bulbasaur presente
S3	Objetos Parquet publicados e baixados com sucesso
Integridade	Hashes local e S3 coincidentes
Segurança	Senha não exibida e ausência de chaves AWS estáticas

3. 3. ARQUITETURA DA SOLUÇÃO

Arquitetura executada no Tutorial 2

Ingestão com Meltano na EC2, geração local de Parquet e publicação no Amazon S3

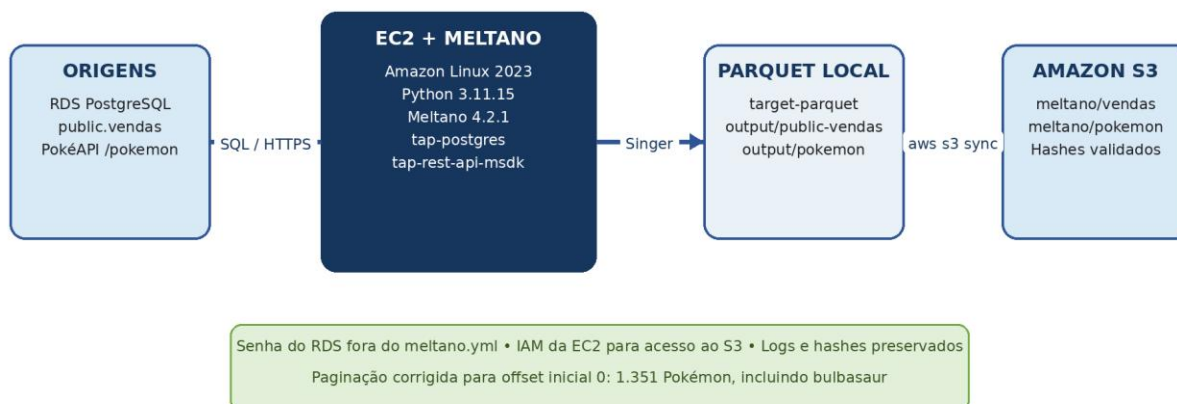


Figura 3-1: Arquitetura executada no Tutorial 2. Fonte: elaboração própria a partir do tutorial e das evidências.

Tabela 3-1: Componentes da arquitetura

Componente	Função	Implementação
Amazon EC2	Ambiente de execução	Amazon Linux 2023; Python 3.11.15; Meltano 4.2.1
Amazon RDS PostgreSQL	Origem estruturada	Banco ecommerce; schema public; tabela vendas
PokéAPI	Origem REST pública	Endpoint /api/v2/pokemon
tap-postgres	Extrator do banco	Variante meltanolabs
tap-rest-api-msdk	Extrator REST	Variante widen; paginação por offset
target-parquet	Conversão e gravação local	Variante automatic; destination_path=output
AWS CLI	Publicação	aws s3 sync para os prefixos meltano/*
Amazon S3	Data lake do laboratório	Parquet de vendas e Pokémon

A separação entre geração local e publicação no S3 foi mantida conforme o tutorial. O target-parquet produziu os arquivos na EC2, enquanto a AWS CLI realizou a transferência para o bucket. Esse desenho tornou possível validar o arquivo local antes do envio e, posteriormente, baixar o objeto remoto para comparação independente.

4. 4. PREPARAÇÃO DA INFRAESTRUTURA E DO AMBIENTE

4.1.Revalidação Da Infraestrutura

A execução iniciou com a validação da sessão AWS, da configuração Terraform e dos recursos existentes. O plano final não apontou alterações, a EC2 estava em execução, o RDS estava disponível e o bucket S3 respondeu ao teste de acesso. A evidência E03 apresenta o resumo da etapa inicial.

```
SessaoAWS      : VALIDADA
TerraformValidate : APROVADO
TerraformSemDrift : SIM
EC2Id          : i-0db119c4f19fd868e
EC2Estado      : running
EC2IpPrivado   : 172.31.35.18
EC2SecurityGroup : sg-0be8efd12443a2a59
RDSId          : ingestao-postgres-20260729181449897700000004
RDSEstado      : available
RDSEndpoint    : ingestao-postgres-20260729181449897700000004.csoa8fqdbrgu.us-east-1.rds.amazonaws.com
RDSBanco       : ecommerce
RDSUsuario     : ecommerce
RDSSecurityGroups : sg-0be8efd12443a2a59
S3Bucket       : 650379606721-ingestao-lab-20260729181445719900000002
S3Acessivel    : True
SenhaExibida   : NAO
Resultado      : APROVADO
```

Figura 4-1: Resumo da validação inicial da infraestrutura. Fonte: evidência E03.

4.2.Instalação Do Python E Do Meltano

A EC2 não possuía inicialmente todos os componentes necessários. Foram instalados Python 3.11, pip, suporte a ambiente virtual e Git. Em seguida, foi criado o ambiente /home/ec2-user/meltano-venv e instalada a versão 4.2.1 do Meltano.

- Comandos centrais de preparação do ambiente

```
python3.11 -m venv ~/meltano-venv
source ~/meltano-venv/bin/activate
pip install --upgrade pip
pip install "meltano==4.2.1"
meltano --version
```

O ambiente virtual foi mantido separado do sistema operacional e precisava ser reativado em cada nova sessão. Essa prática reduziu conflitos de dependências e permitiu registrar com precisão as versões utilizadas.

5. CRIAÇÃO E CONFIGURAÇÃO DO PROJETO MELTANO

5.1. Projeto E Plugins

O projeto foi criado no diretório `/home/ec2-user/projeto_ingestao`. O envio de estatísticas anônimas foi desabilitado e os três plugins foram instalados. Para o `tap-rest-api-msdk`, o `pip_url` foi ajustado com `setuptools<81`, devido à compatibilidade do pacote com a API `pkg_resources`.

Tabela 5-1: Plugins e configurações principais

Plugin	Variante	Finalidade	Configuração relevante
tap-postgres	meltanolabs	Extrair public.vendas	RDS, porta 5432, usuário e database ecommerce
tap-rest-api-msdk	widen	Consultar /pokemon	offset, página 100, records_path \$.results[*]
target-parquet	automattic	Gerar Parquet local	destination_path: output

5.2. Configuração Segura

O `meltano.yml` armazenou somente parâmetros não sigilosos, como endpoint, porta, usuário e banco. A senha do RDS foi fornecida por variável específica no arquivo `.env`, protegido e excluído das evidências publicáveis. Não foram identificadas chaves AWS no pacote.

- Trecho simplificado da configuração final

```
plugins:
  extractors:
    - name: tap-postgres
      variant: meltanolabs
      select:
        - public-vendas.*
```

```
- name: tap-rest-api-msdk
  variant: widen
  config:
    pagination_page_size: 100
    pagination_initial_offset: 0
loaders:
- name: target-parquet
  variant: automattic
  config:
    destination_path: output
```

6. INGESTÃO DO RDS POSTGRESQL

6.1.Descoberta E Seleção

O tap-postgres foi configurado para o endpoint do RDS e a seleção public-vendas.* restringiu a extração à tabela requerida. Antes da execução efetiva, foram preservados o catálogo descoberto e o dry run, permitindo verificar o stream e a integração com o target-parquet.

6.2.Execução E Validação

A pipeline meltano run tap-postgres target-parquet produziu um arquivo Parquet no diretório output/public-vendas. A leitura com pyarrow confirmou exatamente 200 linhas, em conformidade com o seed da infraestrutura.

Tabela 6-1: Resultado da ingestão PostgreSQL

Indicador	Resultado
Stream	public-vendas
Quantidade de Parquet	1
Total de linhas	200

Indicador	Resultado
Total esperado	200
Tamanho do arquivo	4.785 bytes
Hash SHA-256 registrado	Sim
Validação	APROVADO

O log interno automático do Meltano não foi localizado no caminho pesquisado. Essa ausência não comprometeu a rastreabilidade, pois a saída integral da execução foi redirecionada para `E23_execucao_pipeline_postgres.log`, acompanhada de catálogo, dry run, validação e hash.

7. INGESTÃO DA POKÉAPI

7.1. Configuração Da Api E Divergência De Quantidade

A PokéAPI foi configurada no `tap-rest-api-msdk` com paginação `simple_offset_paginator`, páginas de 100 registros e extração a partir de `$.results`. Embora o tutorial utilizasse 1.350 como referência, a consulta realizada em 30 de julho de 2026 informou e retornou 1.351 registros.

7.2. Diagnóstico E Correção Do Offset

A primeira execução gerou 1.350 registros e omitiu `bulbasaur`. A comparação nominal entre a lista retornada pela API e os nomes presentes no Parquet identificou exatamente um registro ausente. O problema decorreu do offset inicial 1 do plugin, enquanto a PokéAPI inicia sua coleção no offset 0.

A configuração `pagination_initial_offset` foi alterada para 0. Inicialmente o Meltano gravou o valor como texto, `pagination_initial_offset: '0'`, provocando falha de validação de tipo. O `meltano.yml` foi então corrigido para o inteiro 0, sem aspas. Após a reexecução, os 1.351 nomes foram carregados e `bulbasaur` passou a constar no Parquet.

Tabela 7-1: Resultado da ingestão da PokéAPI

Indicador	Resultado
Total previsto no tutorial	1.350
Total informado e retornado pela API	1.351
Quantidade de Parquet	1
Total de linhas no Parquet	1.351
Nomes únicos	1.351
Bulbasaur presente	Sim
Diferença em relação ao tutorial	+1
Validação	APROVADO

8. PUBLICAÇÃO E VALIDAÇÃO NO AMAZON S3

8.1.Publicação

Os arquivos locais foram publicados com aws s3 sync. Antes de cada envio, o prefixo correspondente foi limpo para evitar que resultados de tentativas anteriores interferissem na contagem. Foram enviados somente arquivos com extensão .parquet.

- Publicação dos arquivos no S3

```
aws s3 sync output/public-vendas s3://$BUCKET/meltano/vendas/ --exclude "*"
--include "*.parquet"
aws s3 sync output/pokemon s3://$BUCKET/meltano/pokemon/ --exclude "*" --
include "*.parquet"
```

8.2.Validação Independente

Cada prefixo foi listado e sincronizado de volta para um diretório de validação na EC2. Os hashes SHA-256 do arquivo local e do objeto baixado foram comparados. Em seguida, o conteúdo Parquet

baixado do S3 foi lido novamente para confirmar as quantidades e, no caso da PokéAPI, a unicidade e a presença de bulbasaur.

Tabela 8-1: Validação dos objetos publicados no S3

Dataset	Prefixo S3	Parquet	Linhas	Hash	Resultado
Vendas	meltano/vendas/	1	200	Coincidente	APROVADO
Pokémon	meltano/pokemon/	1	1.351	Coincidente	APROVADO

```

=====
RESUMO DA ETAPA 11
=====
Data=2026-07-30T17:54:55+00:00
Origem=/home/ec2-user/projeto_ingestao/output/pokemon
Destino=s3://650379606721-ingestao-lab-20260729181445719900000002/meltano/pokemon
ArquivosOrigem=1
CleanS3ExitCode=0
SyncS3ExitCode=0
ListagemS3ExitCode=0
ArquivosParquetS3=1
TotalBytesS3=16185
DownloadS3ExitCode=0
HashesIguais=SIM
TotalLinhasS3=1351
TotalNomesUnicosS3=1351
BulbasaurPresentesS3=SIM
ContentValidationExitCode=0
ResultadoConteudo=APROVADO
TotalEsperadoAPI=1351
TotalPrevistoTutorial=1350
Resultado=APROVADO
(meltano-venv) [ec2-user@ip-172-31-35-18 projeto_ingestao]$ S3_EVID="s3://$BUCKET/evidencias/tutorial2/08-publicacao-s3"

```

Figura 8-1: Resumo da publicação e validação da PokéAPI no S3. Fonte: evidência E50.

9. SEGURANÇA, RASTREABILIDADE E EVIDÊNCIAS

A solução evitou a inclusão da senha no meltano.yml e não utilizou chaves AWS estáticas. O acesso ao S3 ocorreu a partir da EC2, aproveitando o perfil IAM do laboratório. O pacote foi inspecionado e não foram identificados padrões de access keys nem atribuições contendo senhas reais.

As evidências foram organizadas por etapa: preparação, instalação do Meltano, criação do projeto, ingestão PostgreSQL, ingestão PokéAPI, publicação no S3, validação final e dificuldades. Os arquivos incluem logs, configurações seguras, capturas de tela, validações, inventário e hashes SHA-256.

O inventário E53 e o arquivo de hashes E54 registram o estado do conjunto de evidências durante a consolidação. Além disso, o pacote final preserva tanto os resultados aprovados quanto as falhas intermediárias que motivaram correções.

10. DIFICULDADES ENCONTRADAS E SOLUÇÕES ADOTADAS

10.1. Python 3.11 Ausente

Os pacotes necessários foram instalados com DNF e o suporte a venv foi testado antes da instalação do Meltano.

10.2. Ambiente Virtual Não Persistiu

A ativação realizada dentro de um pipeline não afetou a sessão principal. O ambiente foi reativado diretamente com source.

10.3. Caractere De Exclamação Na Senha

A expansão de histórico do Bash provocou erro. A senha foi tratada por entrada protegida e armazenamento controlado.

10.4. Conexão Com EC2 Incorreta

A identidade da instância foi conferida com IMDSv2 e a conexão foi refeita para i-0db119c4f19fd868e.

10.5. Divergência Da Senha Do RDS

A credencial foi redefinida e sincronizada com o Terraform, após o que o plano retornou sem alterações.

10.6. Pokéapi Com 1.351 Registros

O total atual da fonte foi adotado para validação, mantendo-se a referência de 1.350 documentada para comparação.

10.7. Bulbasaur Ausente

A comparação nominal identificou o primeiro registro omitido; o offset inicial foi corrigido de 1 para 0.

10.8. Offset 0 Gravado Como Texto

O valor foi corrigido diretamente no YAML para um inteiro, eliminando a falha de validação do plugin.

10.9. Log Interno Não Localizado

A saída das execuções foi preservada por redirecionamento e logs próprios, suficientes para auditoria.

10.10. Variáveis Vazias Em Blocos Parciais

As validações foram refeitas com scripts independentes e captura explícita dos códigos de saída.

O conjunto completo dessas ocorrências está consolidado em E52_dificuldades_e_solucoes.txt. A preservação dos problemas e das soluções permite demonstrar não apenas o resultado final, mas também a evolução técnica da execução.

11. AVALIAÇÃO DE CONFORMIDADE COM O TUTORIAL

Tabela 11-1: Matriz de conformidade com o Tutorial 2

Requisito do tutorial	Implementação realizada	Situação
Executar Meltano na EC2	Ambiente meltano-venv na instância runner	ATENDIDO
Extrair public.vendas do RDS	tap-postgres com seleção public-vendas.*	ATENDIDO
Consultar a PokéAPI	tap-rest-api-msdk no stream pokemon	ATENDIDO
Gerar Parquet	target-parquet em output/public-vendas e output/pokemon	ATENDIDO

Requisito do tutorial	Implementação realizada	Situação
Publicar no S3 real	aws s3 sync para meltano/vendas e meltano/pokemon	ATENDIDO
Usar IAM sem chaves	Operações S3 realizadas a partir da EC2	ATENDIDO
Validar objetos no S3	Listagem, download, leitura e hashes	ATENDIDO E AMPLIADO
Esperar 1.350 Pokémon	Fonte atual retornou 1.351, todos validados	ATUALIZADO
Configuração de paginação	Offset inicial corrigido para 0	APERFEIÇOADO
Destruir infraestrutura ao final	Postergado até a revisão e entrega	PENDENTE CONTROLADO

A postergação do terraform destroy é controlada. Como o bucket foi configurado com `force_destroy`, a remoção da infraestrutura apagará também os Parquet e as evidências remotas. A destruição deve ocorrer somente após a conferência do relatório e a guarda local dos arquivos necessários para entrega.

12. CONCLUSÃO E PRÓXIMOS PASSOS

O Tutorial 2 foi concluído com êxito. O Meltano foi instalado em ambiente isolado, os plugins foram configurados e as duas ingestões foram executadas. A carga do RDS produziu 200 registros, enquanto a PokéAPI produziu 1.351 registros únicos, incluindo bulbasaur.

A publicação no S3 foi comprovada por listagens, downloads de validação, leitura dos Parquet e comparação de hashes. O tratamento de segredos e a autenticação por IAM evitaram a exposição de credenciais no código e nas evidências.

As correções realizadas durante o trabalho, especialmente no tratamento da paginação da PokéAPI e do tipo do offset no YAML, demonstram a importância de validar o conteúdo dos dados, e não apenas o sucesso formal do comando de execução.

12.1. Próximos Passos Recomendados

- Revisar o relatório e armazená-lo junto ao pacote de evidências.

- Publicar no GitHub somente códigos, configurações sem segredos, relatório e evidências selecionadas.
- Manter fora do repositório o arquivo .env, credenciais AWS e qualquer arquivo de senha.
- Após a entrega, executar terraform destroy e confirmar a remoção de EC2, RDS e S3 para evitar custos.

13. APÊNDICE A – MAPA DAS EVIDÊNCIAS

Tabela 13-1: Evidências selecionadas para rastreabilidade

Evidência	Pasta	Conteúdo
E00–E03	00-preparacao	Diagnóstico, resumo, plano Terraform e captura da infraestrutura
E06–E10	03-instalacao-meltano	Instalação de Python/Meltano e reativação do ambiente
E11–E19	04-projeto-meltano	Criação do projeto, plugins e configurações iniciais
E20–E26	05-ingestao-postgres	Configuração, catálogo, dry run, execução, validação e hash
E34–E44	06-ingestao-pokeapi	Configuração, execução corrigida, validação, hash e YAML final
E29–E33	08-publicacao-s3	Publicação e validação das vendas no S3
E45–E50	08-publicacao-s3	Publicação e validação da PokéAPI no S3
E51, E53–E55	09-validacao-final	Listagem, inventário, hashes e resumo final
E52	10-dificuldades	Dificuldades e soluções adotadas

A relação completa dos arquivos está no inventário E53_inventario_evidencias.csv. O arquivo E54_hashes_evidencias_sha256.csv registra hashes dos documentos existentes no momento de sua geração e permite apoiar verificações de integridade.

14. APÊNDICE B – NOTA SOBRE A CONTAGEM DE ARQUIVOS DO PACOTE

O resumo E55 registra 50 arquivos porque a contagem foi realizada antes da criação do próprio E55. O pacote ZIP final contém 51 arquivos. A diferença representa somente a inclusão posterior do resumo final e não indica ausência de evidências.

As contagens intermediárias do inventário e dos hashes também refletem a sequência de geração dos arquivos. O inventário E53 possui 48 registros e o arquivo E54 possui 49 hashes. Posteriormente, os documentos finais foram adicionados ao conjunto, resultando nos 51 itens efetivamente presentes no ZIP.

A inspeção do pacote confirmou a presença dos arquivos listados e não identificou chaves AWS nem atribuições de senha real. O arquivo .env utilizado na execução não integra o pacote.